

Scripting

In

Python

Junaid Khan

23-09-2025

Introduction

Python allows us to access our systems directory structure, set the environment variables and manipulate the file system. Later, we can create a script out of different operations to automate the process.

Uses three main python packages to perform file operations:

1. os library guide
2. shutil library
3. Path from pathlib

os library

The os library is used to request services from the operating system. These can help us to create or remove directories/directory trees, create new paths, rename/list files in a certain folder or check if a certain file/folder exists and extract the file extension.

Import the os module using: `import os`

```
>>> import os
>>> os.getcwd()
'C:\\Users\\junaid.khan'
```

- `print(os.getcwd())` lists the current directory path.
- `os.chdir('dest_dir')`, changes the directory to the desired one. Python uses `'\\'` as escape character. Don't forget to add double back-slash in path name.
- `os.listdir()` lists the files in the current directory. This returns a python list of all the files and folders in that directory.

```
>>> os.chdir('F:\\PD')
>>> os.listdir()
['65nm_LP_STD_Cells', 'AN_N65_SRAM_Compiler_Installation_UserGuide_V3_wmc.pdf',
```

You can rename a file/folder using the `os.rename('old_name', 'new_name')` command.

There are two commands to create a directory:

- `os.mkdir('dir_name')`; This creates a level 1 directory.
- `os.makedirs('dir1_name/dir2_name')`; create a directory tree or sub-directories under the top dir.

...

- To delete a directory path recursively, use `os.removedirs('dir_path')`. The command `os.rmdir('dir_name')` will not perform deletion until the directory is empty.
- In-order to create a full path name, use path module inside the os library instead of string concat which often misses certain backslashes such as: `full_path = os.path.join(os.getcwd(), 'new_file.txt')`.

```
>>> full_path = os.path.join(os.getcwd(), 'new_file.txt')
>>> print(full_path)
F:\PD\new_file.txt
```

- In above example `full_path` is a string that stores an absolute path from root and the `new_file.txt` file.
- The path module functions are very useful to check if a valid path exists or to split the `path_name` and `file_name` by returning a tuple. This shows false since there is no 'file.txt' in `file_name`.

- Split the `path_name` and `file_name`: `os.path.split(full_path)`

```
>>> os.path.exists('full_path')
False
```

- We can also extract the `file_extension` using `os.path.splitext(full_path)`. This is useful when sorting files based on extension.

```
>>> os.path.split(full_path)
('F:\\PD', 'new_file.txt')
>>> |
```

```
>>> os.path.splitext(full_path)
('F:\\PD\\new_file', '.txt')
>>> |
```

- To check if an item is a file or a directory, append 'isdir' or 'isfile' option in-front of `os.path.?( 'file_path')`.

...

- To get the value of stored environment variables, use `os.environ.get('name_of_variable')`. You can then use them to append to the path names.
- Another useful function is `os.walk('dir_path')`. This returns a list of `path_names`, `dir_names` and `file_names` from the root path which can later be iterated upon.

```
>>> os.environ.get('HOMEPATH')  
'\\Users\\junaaid.khan'
```

```
>>> for path_name, dir_name, file_name in os.walk('C:\\Users\\junaaid.khan\\Desktop\\pyrhon'):  
...     print(path_name)  
...     print(dir_name)  
...     print(file_name)  
...  
C:\\Users\\junaaid.khan\\Desktop\\pyrhon  
['destination_dir', 'unpack_dir']  
['another_dir.zip', 'app.py', 'file.txt']  
C:\\Users\\junaaid.khan\\Desktop\\pyrhon\\destination_dir  
[]  
['app.py', 'file.txt']  
C:\\Users\\junaaid.khan\\Desktop\\pyrhon\\unpack_dir  
[]  
['app.py', 'file.txt']  
>>>
```

- To inquire file statistics, eg. date of modification or size in bytes, use `os.stat('file_path')`.

```
>>> os.stat('Macros.zip')  
os.stat_result(st_mode=33206,
```

- Alternatively, you can set an environment variable value using `os.environ['var_name']="value"`.

shutil library

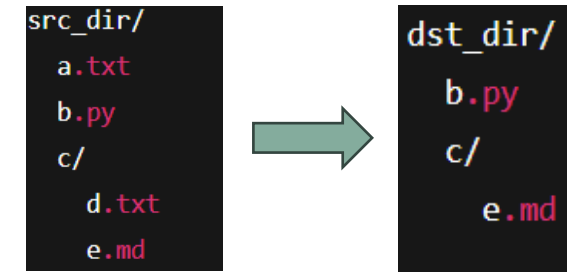
- This library completes the os library by providing additional functions for copying, moving or deleting directories. Additionally, you can create new functions that serve the existing library routines. Import the library first in your program.

```
>>> shutil.copy('Macro_Integration_Tutorial.pdf', 'C:\\Users\\junaid.khan\\Desktop\\pyrhon')
'C:\\Users\\junaid.khan\\Desktop\\pyrhon\\Macro_Integration_Tutorial.pdf'
```

- The `shutil.copy(src='file_1.txt', dst='dest_dir')` command copies the “src” file to a destination directory.
- To copy a directory structure from one location to another recursively, use `shutil.copytree(src='source_dir', dst='dest_dir')`. Include `dirs_exist_ok=True` argument if destination directory already exists.
- The `copytree` function receives an additional `ignore` argument that lets it to exclude the files when traversing the directories.

```
def ignore_files(directory, files):
    # Ignore all .txt files
    return [f for f in files if f.endswith('.txt')]

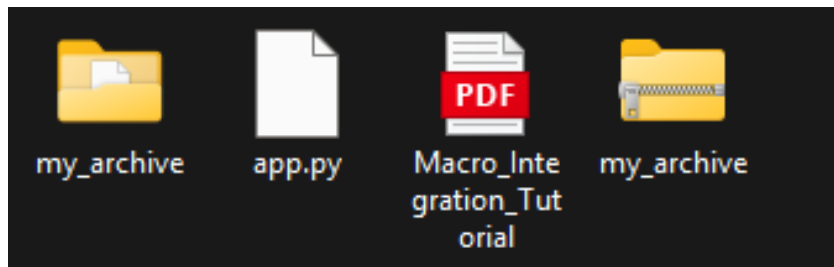
shutil.copytree(src='src_dir', dst='dst_dir', ignore=ignore_files)
```



- Instead of copying, `shutil.move('src_file', 'des_dir')` moves the file.
- To delete a directory which is not empty, use `shutil.rmtree('directory_path')`.

...

- Shutil library contains functions for generating and un-packing the archive files. To create an archive, use `shutil.make_archive('archive_name', 'format', 'folder_path')`. Format here can be 'zip' or any others supported by the function.
- Use `get_archive_formats()` to enlist archive formats.
- The `shutil.unpack_archive('archive_path.format', 'extraction_folder')` function basically extracts the archived files.



```
>>> shutil.make_archive('my_archive', 'zip', 'my_folder')  
'C:\\Users\\junaaid.khan\\Desktop\\pyrhon\\my_archive.zip'
```

```
>>> shutil.unpack_archive('my_archive.zip', 'my_archive')
```

pathlib guide

- Pathlib provides a similar functionality as the os module, however, its advantage lies in the fact that it considers the path items as objects rather than strings that the os module is accustomed to.
- You can create a pathlib object from a file/directory. In your python script; use from pathlib import Path to incorporate the path module in your script.
`my_dir = Path("directory_path/file_path")`
- The above expression creates a path object and stores it in the my_dir variable. Path.cwd() function returns the current directory path.
- Path.home() gives path to the home directory. The 'parent' attribute of the path object gives a relative path to that path's parent folder. To obtain an absolute path, use the absolute() or resolve() method in-front of the path object.
- The resolve() function tries to solve any symbolic links such as '..' and evaluates to the correct expression
- You can check whether a directory exists or not using the exists() method.
- Paths can usually be joined using the forward slash operator "/".
- If you want to extract the extension of a path file, use the suffix attribute.

```
>>> my_dir = Path.cwd()
>>> print(my_dir)
C:\Users\junaaid.khan\Desktop\pyrhon
```

```
>>> Path.home()
WindowsPath('C:/Users/junaaid.khan')
>>> print(my_dir)
C:\Users\junaaid.khan\Desktop\pyrhon
>>> print(my_dir.parent)
C:\Users\junaaid.khan\Desktop
>>> print(my_dir.resolve())
C:\Users\junaaid.khan\Desktop\pyrhon
>>> print(my_dir.exists())
True
```

```
>>> my_dir = my_dir / "app.py"
>>> print(my_dir.suffix)
.py
>>> print(my_dir.stem)
app
>>> |
```

...

- Another important function in the library is the `glob("search_pattern")` which prints all those files in the path directory that match the search pattern. For recursive searching, use the `rglob("search_pattern")` method.

```
>>> for p in curr_dir.rglob("*.vscode", case_sensitive=False):  
...     print(p)  
...  
C:\Users\junaaid.khan\.vscode
```

```
>>> with file.open() as f:  
...     content = f.read()  
...  
>>> print (content)  
ID = 24 ANSWER SAME AS ID = 25  
>>> |
```

- You can use the `path.open()` method to read a file's content.
- To delete a file use the `unlink()` method as such.
- Additionally, a path object directory can be removed if `path.rmdir()` is used with it.
- `Path.touch()` will create a new path file.
- To rename a file, `replace/rename` methods of the `pathlib` library can be used.

References:

- Python shutil module: <https://docs.python.org/3/library/shutil.html>
- Python pathlib module: <https://docs.python.org/3/library/pathlib.html>